

DSA FROM SCRATCH TO ADVANCED

Day 10

Binary Search — Introduction

Concept → Example → Implementation → Complexity → Pattern → Problems → Takeaway

LinkedIn Learning Series

A practical, beginner-friendly guide to understanding one of the most important searching techniques in DSA.

Learning Roadmap

Today's goal is not just to memorize binary search. The goal is to understand the condition that makes it possible, trace the algorithm manually, implement it safely in C++, and recognize the pattern in interview questions.

- Understand what binary search is.
- Know why sorted data changes the searching strategy.
- Learn the left, right, and mid pointers.
- Trace binary search step by step.
- Implement iterative binary search in C++.
- Analyze time and space complexity.
- Recognize common interview patterns.
- Practice with LeetCode problems.

Core progression: Concept → Example → Implementation → Complexity → Pattern → Problem → Solution → Takeaway

1. What Is Binary Search?

Binary Search is a searching algorithm used primarily on a sorted array or sequence. Instead of checking every element from left to right, it repeatedly checks the middle element and eliminates half of the remaining search space.

If the target is smaller than the middle value, the right half can be discarded. If the target is larger, the left half can be discarded.

Key idea: Every comparison can remove approximately half of the remaining candidates.

Example sorted array: [2, 5, 8, 12, 16, 23, 38, 56, 72]

2. Why Do We Need Binary Search?

Linear Search is simple, but it may inspect every element. For a large array, this can become expensive.

Method	Requirement	Worst-case time
Linear Search	No sorting required	$O(n)$
Binary Search	Sorted / ordered search space	$O(\log n)$

For 1,000,000 sorted elements, linear search can require around 1,000,000 checks in the worst case, while binary search needs only about 20 comparisons because 2^{20} is greater than 1,000,000.

3. The Most Important Requirement

Binary search needs an ordered search space.

For a normal ascending sorted array, we know that everything to the left of the middle is less than or equal to the middle, while everything to the right is greater than or equal to it.

- Ascending order: $a[mid]$ separates smaller and larger values.
- Descending order: the comparison direction changes.
- Unsorted data: standard binary search is not directly applicable.

This is why binary search is more than an algorithm: it is a pattern for eliminating impossible regions.

4. How Does It Work?

Maintain three variables:

- left = beginning of search range
- right = end of search range
- mid = middle index

At each iteration:

- Calculate mid.
- If $a[mid] == target \rightarrow$ answer found.
- If $a[mid] < target \rightarrow$ search the right half.
- If $a[mid] > target \rightarrow$ search the left half.
- Stop when the search interval becomes empty.

The search interval keeps shrinking by roughly half.

5. Step-by-Step Example

Target = 23

Array = [2, 5, 8, 12, 16, 23, 38, 56, 72]

- left = 0, right = 8, mid = 4 → value = 16.
- $16 < 23$, so discard indices 0-4.
- left = 5, right = 8, mid = 6 → value = 38.
- $38 > 23$, so discard indices 6-8.
- left = 5, right = 5, mid = 5 → value = 23.
- Target found at index 5.

Notice: We checked only three middle positions instead of scanning the entire array.

6. Visual Search Pattern

Think of the array as a search window:

[LEFT MID RIGHT]

After comparing with MID, one entire side becomes irrelevant.

Case A: $\text{target} == \text{mid} \rightarrow \text{return mid}$

Case B: $\text{target} > \text{mid} \rightarrow \text{LEFT moves to mid} + 1$

Case C: $\text{target} < \text{mid} \rightarrow \text{RIGHT moves to mid} - 1$

This shrinking-window idea is the foundation of many binary-search problems.

7. C++ Implementation — Iterative

```
int binarySearch(vector<int>& arr, int target) {  
    int left = 0;  
    int right = arr.size() - 1;  
  
    while (left <= right) {  
        int mid = left + (right - left) / 2;  
  
        if (arr[mid] == target)  
            return mid;  
  
        if (arr[mid] < target)  
            left = mid + 1;  
        else  
            right = mid - 1;  
    }  
  
    return -1;  
}
```

Returning -1 is a common convention when the target does not exist.

8. Why Calculate mid This Way?

```
int mid = left + (right - left) / 2;
```

You will often see $(\text{left} + \text{right}) / 2$. It works for many normal inputs, but the addition can overflow when indices are very large.

Using $\text{left} + (\text{right} - \text{left}) / 2$ avoids that particular overflow risk.

Interview habit: Prefer the safer midpoint expression.

9. Dry Run — Target Found

Array = [3, 7, 11, 15, 19, 24, 31], target = 15

Iteration	left	right	mid	a[mid]	Decision
1	0	6	3	15	Found

The first middle element happens to be the target, so the algorithm terminates immediately.

10. Dry Run — Target Not Found

Array = [2, 4, 6, 8, 10, 12], target = 9

- mid = 2 → value 6. Target is larger → search right.
- mid = 4 → value 10. Target is smaller → search left.
- mid = 3 → value 8. Target is larger → search right.
- left becomes 4 and right becomes 3.
- left > right → search space is empty → return -1.

This termination condition is essential. It prevents an endless loop.

11. Time & Space Complexity

Time Complexity: $O(\log n)$

Each step roughly halves the remaining search space:

$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots \rightarrow 1$

The number of halvings needed to reach 1 is logarithmic.

Space Complexity: $O(1)$ for the iterative implementation because only a few variables are maintained.

Recursive binary search typically uses $O(\log n)$ auxiliary stack space.

12. Binary Search vs Linear Search

Feature	Linear Search	Binary Search
Data order	Any	Sorted / ordered
Approach	Check sequentially	Eliminate half
Worst-case	$O(n)$	$O(\log n)$
Implementation	Very simple	Slightly more error-prone
Best use	Small/unsorted data	Large sorted search space

Binary search is not automatically better: if the data is unsorted and you only search once, sorting first may cost more than simply scanning.

13. The Binary Search Pattern

A powerful way to recognize binary search is to ask:

- Is the search space ordered?
- Can I decide which half is impossible after one comparison?
- Does a monotonic condition exist?
- Can the answer be described as the first/last position where a condition becomes true?

If the answer is yes, binary search may be the right pattern—even when the problem does not literally say 'search an array'.

14. Common Variants

- Find an exact target.
- Find the first occurrence of a value.
- Find the last occurrence of a value.
- Lower bound: first position where value $\geq$ target.
- Upper bound: first position where value $>$ target.
- Search in a rotated sorted array.
- Binary search on the answer: search a numeric range using a feasibility condition.

These variants are where binary search becomes an important interview pattern rather than a single memorized algorithm.

15. Common Mistakes

- Applying standard binary search to an unsorted array.
- Using incorrect boundary updates.
- Writing $\text{mid} = (\text{left} + \text{right}) / 2$ without considering overflow.
- Mixing inclusive and exclusive interval conventions.
- Forgetting to move left/right after checking mid.
- Using the wrong comparison for descending arrays.
- Not testing target smaller than all values or larger than all values.
- Not handling duplicate values correctly when the problem asks for first/last occurrence.

Best practice: Choose one interval convention and use it consistently.

16. LeetCode Practice — Beginner

Problem 1 — Binary Search

LeetCode 704: Given a sorted array and a target, return the index if the target exists; otherwise return -1.

What to practice: Basic left/right/mid logic.

Problem 2 — Search Insert Position

LeetCode 35: Return the index where target is found or where it should be inserted to keep the array sorted.

What to practice: Binary search when the exact value may not exist.

17. LeetCode Practice — Pattern Building

Problem 3 — First Bad Version

LeetCode 278: Find the first bad version using a monotonic true/false condition.

What to practice: Binary search on a condition rather than directly searching values.

Recommended progression:

- 704 — Binary Search
- 35 — Search Insert Position
- 278 — First Bad Version

After these, move toward lower/upper bounds, rotated arrays, and binary search on answer.

18. Interview Tip

Interview Tip: Don't immediately start coding binary search.

First state the invariant and interval clearly:

"I will search the inclusive range [left, right]. At every step, I compare the target with $a[mid]$ and eliminate the half that cannot contain the answer."

Then explain why the discarded half is impossible. This demonstrates understanding instead of memorization.

If duplicates or first/last occurrence are involved, explicitly state how your boundary update changes.

19. Mini Interview Question

Question: Why is binary search $O(\log n)$?

Answer: Because each comparison reduces the remaining search space by approximately half. After k steps, roughly $n / 2^k$ elements remain. We stop when that becomes 1, so k is proportional to $\log_2(n)$.

Follow-up: Can binary search be used on something that is not an array?

Yes. It can work on any ordered/monotonic search space where a decision lets us safely eliminate one side.

20. Real-World Analogy

Imagine searching for a name in a dictionary.

- Opening near the middle is faster than starting from page 1.
- If the name alphabetically comes later, ignore the earlier half.
- Open the middle of the remaining pages.
- Repeat until the range becomes very small.

That is binary search in everyday life: use ordering to eliminate large portions of the search space.

21. Quick Revision Sheet

- Binary Search → search an ordered space.
- Core variables → left, right, mid.
- Found → return mid.
- Target greater → $\text{left} = \text{mid} + 1$.
- Target smaller → $\text{right} = \text{mid} - 1$.
- Loop for inclusive range → while ($\text{left} \leq \text{right}$).
- Time → $O(\log n)$.
- Iterative space → $O(1)$.
- Safer mid → $\text{left} + (\text{right} - \text{left}) / 2$.
- Main pattern → eliminate half of the search space.

One-line takeaway: Binary search is about reducing the problem size aggressively by using order or monotonicity.

22. Takeaway + Tomorrow

Today's takeaway

- You learned the basic binary search algorithm.
- You learned why sorted data matters.
- You implemented it in C++.
- You analyzed $O(\log n)$ time and $O(1)$ iterative space.
- You saw how binary search evolves into a reusable problem-solving pattern.

What I'll learn tomorrow:

Day 11 — Binary Search: Lower Bound, Upper Bound & First/Last Occurrence

The next step is to move from exact searching to boundary-based searching—an essential interview pattern.

Practice: LeetCode 704, 35, and 278.

23. Series Closing

DSA From Scratch to Advanced — Day 10

Binary Search is one of those topics that looks simple in code but teaches a powerful problem-solving idea: eliminate what cannot be the answer.

Keep practicing the pattern, trace every boundary change, and focus on why each half can be discarded.

GitHub — Full DSA PPT/PDF Resources

https://github.com/ritesh2687/dsa_ppt_post

Follow the series for the next topic.